ShelfSense AI · Judges' Q&A

Data first: why synthetic shelves, what real datasets exist, and what our numbers mean. Then the other questions judges are likely to ask.

The 30-second answer: why synthetic data?

> "To measure a compliance system, every test image needs an **answer key**: which slot is empty, which price is wrong, which product is misplaced. Public shelf datasets don't have that. SKU-110K, the largest, has about 11,700 real shelf photos, but every box is labelled just 'product': no product identity, no price tags, no planograms, no labelled violations. So we built a generator that draws shelves from the planogram, plants mistakes on purpose and records the answers. We trained and tested on **2,800 synthetic shelves with about 108,000 labelled boxes**, so we can report exact accuracy. The pipeline is built so real photos and SKU-110K plug in with the same commands."

Real datasets that exist, and what they lack

Dataset	Size	What it gives	What it lacks for ShelfSense
SKU-110K (CVPR 2019)	8,219 train / 588 val / 2,936 test images; ~147 products per image	Dense, real supermarket shelf photos	One class only ('product'); no product identity, price tags, gaps or planograms; non-commercial research licence
Grozi-3.2k	~3,200 shelf images	Swiss supermarket shelves + reference product photos	Swiss products; no ₹ tags; no labelled violations
RP2K	500,000+ images, 2,000 products	Real in-store product photos, natural light	Single products, not full shelves with tags
Unitail	1,454 products, 30,000 text regions	Reading text on packaging (OCR)	Price tags not confirmed; no planograms
Grocery Shelves (UniData)	5,000+ images	Labels for facing, flipped, hidden items	Commercial (paid) dataset

We found **no public dataset** with Indian products and ₹ price tags, planograms, or labelled compliance violations.

What ShelfSense needs vs what real datasets provide

ShelfSense needs	Available in real datasets?
Product bounding boxes	Yes: SKU-110K, Grozi-3.2k
Which product it is	Partly: Grozi-3.2k, RP2K (not Indian brands)
Price tags with readable prices	No
Empty-gap labels	No
Planograms (how the shelf should look)	No
Answer key of mistakes (wrong price, misplaced, out of stock)	No

Our synthetic dataset

Split	Images	Products	Price tags	Empty gaps	Used for
Train	2,000	51,142	18,500	2,255	Training the detector
Val	300	7,688	2,775	327	Checking progress; calibrating the 'unknown product' threshold
Test	500	12,830	4,625	545	Final exam, never seen in training
Total	**2,800**	**71,660**	**25,900**	**3,127**	**about 108,000 labelled boxes**

- 12 Indian products (noodles, biscuits, chips), 4 shelf layouts (planograms), 7 mistake types: out of stock, low stock, misplaced, unknown product, product from another bay, wrong price, wrong label.
- Camera effects: tilt, perspective, exposure, colour cast, glare, vignette, focus blur, hand shake, noise, JPEG.
- Rendered at real phone-photo resolution (~1.6–2.4 px per mm, like a 12 MP photo of a 1.2 m bay).
- Generated in about 8 minutes; fixed random seeds, so anyone regenerates exactly the same images.
- Every image comes with YOLO labels and a JSON answer key (slot contents, violations, true score).

Results on the 500 synthetic test shelves

Target (abstract, section 11)	Target	Result
Product detection mAP@0.5	at least 0.85	**0.995** (mAP@0.5:0.95 = 0.975)
SKU identification, top-1	at least 90%	**98.1%**
Price OCR exact match	at least 90%	**97.4%** (tag product read correctly 97.7%)
Empty-gap recall	at least 90%	**99.5%**
Latency per image (4 vCPU, 8 GB, no internet)	at most 3 s	**1.85 s** mean, 2.31 s p90

End to end (photo to audit): **94.8% precision and 94.3% recall** on violations; 84.6% of shelves get exactly the right list; compliance score off by **2.6 points** on average; about 0.7 items per shelf sent to human review. With perfect detections, the rules engine alone gets **99.8%** of shelves exactly right.

Data questions judges are likely to ask

Q. Isn't synthetic data too easy? Your numbers look too good.

Yes, and we say so openly. Our packs are simpler than real packaging, so these numbers are an upper bound. What synthetic data proves is that the **logic** is right: with perfect detections the compliance engine gets 99.8% of shelves exactly right. On real photos, accuracy depends on perception, which is exactly what SKU-110K and real store photos would improve.

Q. Why not just use SKU-110K?

It only says 'a product is here'. It doesn't say which product, has no price tags or empty-gap labels, and has no planogram to compare against, so we can't measure 'found the wrong price' with it. It is in our plan for training the detector on real packaging, alongside our synthetic shelves for tags and gaps.

Q. How do you know it works on real shelves?

We don't claim that yet; it's the next step. The abstract's plan is to measure on about 500 synthetic shelves, then check on a few real phone photos. Most of the system doesn't depend on the images: rules, scoring, the task writer, dashboard and report work the same on real data. Only the detector and the product gallery need real examples, and both retrain with one command.

Q. How many models did you train?

Only one: the YOLO11n detector, fine-tuned from pre-trained weights on 2,000 synthetic shelves for 60 epochs (about an hour on an RTX 4060). The product matcher (DINOv2-small + colour histogram + FAISS) is not trained; it compares against 144 reference images. OCR (PaddleOCR models via RapidOCR) and Qwen2.5-3B are used as they are.

Q. Where is the data, and can we reproduce it?

It's in data/synthetic/train, val, test: images, YOLO labels and JSON answer keys. It isn't stored on GitHub because it's large and fully reproducible: the generator commands in the README use fixed seeds and recreate the same 2,800 images in about 8 minutes.

Q. How do you add a new product?

Add 3–5 photos and rebuild the product gallery (about 3 minutes, no retraining), then add one line to the catalogue CSV and place it in the planogram JSON.

Q. What about different lighting and angles in real stores?

We simulate them: exposure, warm/cool colour casts, glare bands, vignetting, tilt, perspective, blur and noise. The product matcher also adds colour-cast and brightness variation to its reference images. Real stores will still vary more, which is why real-photo validation is our next step.

Q. Is synthetic data an accepted practice?

Yes. When labelled real data doesn't exist, it is a standard way to start. Published work on shelf recognition makes the same point: standard datasets are often region-specific or too generic for planogram compliance.

Q. Why Indian products?

The problem statement targets Indian retail. Prices in ₹ and local brands (Maggi, Parle-G, Lay's, Kurkure) make the demo realistic, and no public dataset covers them, another reason to generate our own.

Q. What would you do with more time?

1. Train the detector on SKU-110K plus synthetic shelves. 2. Photograph 50–100 real shelves at a local store, label them and measure. 3. Use real product photos in the gallery. The pipeline is ready for all three.

Other questions judges may ask

Design and AI

Q. Why do rules make the decisions instead of the AI?

So every flag can be explained and audited. Neural models only see (boxes, product identity, tag text); a deterministic rules engine decides what is wrong, the score and the order of fixes. A manager can trace any flag to one slot on one shelf.

Q. Why a 3B small language model, and why not ChatGPT?

It runs offline on CPU, with no cloud cost and no store photos leaving the device. It only rewrites the engine's findings into plain instructions; it never decides anything. A 3B model (Qwen2.5-3B, 4-bit, 2.1 GB) is enough for that and fits the edge budget.

Q. Can the language model make things up?

Its output is forced into JSON with exactly one task per problem, and every line is checked: it must keep the row and position (and the destination for a move), use the right kind of action and name the right product. The summary may only use numbers and products from the audit. Failing lines fall back to fixed template text. On 30 test shelves, 88% of task lines came from the model, all checked.

Q. How do you know which product is misplaced, and where it belongs?

Each detected row is aligned to the planned row with an edit-distance alignment that also uses position in the photo. A foreign product in a planned slot is reported as misplaced, with its correct row and position looked up in the planogram.

Q. How is the score calculated?

Score = 100 × (0.50 × Placement + 0.25 × Labels + 0.25 × Availability), as in the abstract. Bands: 90 and above Good, 70–89 Needs attention, below 70 Critical. Items sent to human review are left out of the score, and the score states how much of the shelf it covers (e.g. ‘based on 93% of checks’).

Q. What happens with a blurry photo or an uncertain read?

Nothing is guessed. Low-confidence product matches and price reads become ‘Check by hand’ items. For videos the sharpest frame is picked automatically, and blurry photos get a ‘consider retaking’ warning.

Q. How is ‘revenue at risk’ calculated?

Sales velocity × price for each stock-out (and in proportion to missing facings for low stock). Velocities are assumed per product in the catalogue; a store would plug in its own sales data.

Edge, speed and deployment

Q. Does it really run offline on edge hardware?

Inside a Docker container limited to 4 vCPU and 8 GB with networking disabled (the same budget as a Raspberry Pi 5, 8 GB): 1.85 s per audit, 4.2 GB peak memory including Qwen. We simulated the edge budget; we have not yet run it on a physical Raspberry Pi.

Q. How big are the models?

Detector 11 MB (YOLO11n, ONNX), product embedder 24 MB (DINOv2-small, int8 ONNX), gallery 0.3 MB, Qwen2.5-3B 2.1 GB. No PyTorch is needed at run time.

Q. How did you get under 3 seconds?

Product embeddings use an int8 model (1.7× faster), price tags are OCR'd with detection sized for small crops (about 17× faster than the default), and threads are capped inside the container: without that, libraries started 20 threads on a 4-CPU budget and the audit took 5.5 s.

Q. How long does the language model take?

About 26 s per report inside the 4-CPU container, once per audit. The audit itself appears instantly with template tasks; the model's wording replaces it when ready.

Q. How can someone try it?

One command with Docker: docker run … ashishsingh5042/shelfsense:latest, then open localhost:8501. There's also a REST API (POST /audit) and a downloadable PDF report per audit.

Q. Does it scale across stores?

Each store runs its own container; audits are stored per bay with score history. New products need only photos; new bays need only a planogram JSON. The API makes it easy to connect to a store's existing systems.

Impact, honesty and next steps

Q. What's the business value?

Shelf checks go from an occasional manual walk-through to a photo per shelf, taken as often as needed. Every problem comes located, explained, ranked by revenue at risk and paired with an action. Retail research is widely cited as finding about 8% of items out of stock at any moment, with roughly 1 in 3 shoppers buying elsewhere when they meet a stock-out (approximate industry figures, not our measurements).

Q. What's new compared with existing approaches?

Revenue-at-risk ranking; a zero-retrain product catalogue; unknown-product alerts; confidence-gated human review instead of guesses; sharpest-frame video audits; and a small language model whose every line is checked against the audit.

Q. What are the current limitations?

Validated only on synthetic shelves; real-store testing is next. The most-confused products are Hide & Seek vs Good Day. Product names on very blurry tags may be unreadable (sent to review). Edge results come from a CPU-limited container, not yet a physical Raspberry Pi. SKU-110K is planned but not used yet.

Q. How can we verify your claims?

Everything is reproducible: run the Docker image, or regenerate the data with fixed seeds and run the evaluation scripts (shelfsense.eval.perception, .compliance, .slm, .benchmark). 57 automated tests cover the generator, rules, engine, OCR parsing, task-writer checks, API and report.

Don't say

× “It works in real stores.” Say instead: “It's validated on synthetic shelves; real-store testing is next.”

× “We trained on SKU-110K.” We haven't yet; it's planned.

× “99% accurate.” Always add “on synthetic test shelves”.

× “It runs on a Raspberry Pi.” Say: “It runs within a Raspberry-Pi-sized budget (4 vCPU / 8 GB) in Docker.”

Sources for the dataset facts

- SKU-110K: docs.ultralytics.com/datasets/detect/sku-110k · docs.voxel51.com (sku110k)
- RP2K: paperswithcode.com/paper/rp2k-a-large-scale-retail-product-dataset
- Unitail: arXiv 2204.00298, “Unitail: Detecting, Reading, and Matching in Retail Scene”
- Grocery Shelves: unidata.pro/datasets/grocery-shelves
- Grozi-3.2k and shelf-dataset review: openaccess.bilgi.edu.tr
- ShelfSense results: reports/*.json in the project; README tables